

SMART CONTRACT AUDIT REPORT

Produced for **DZap**
by **Null Return**

Date: **June 2025**

PUBLIC

Contents

Prepared by	1
Introduction	1
About DZap	2
Project Overview	2
Scope	2
Methodology	3
Risk Classification	4
Findings	4
High	4
Medium	4
Low	4
Informational	5
Severity Levels	5
Disclaimer	5
Recommendations	5
About Null Return	6

Prepared by

Project Manager: Nataly Demidova **Email:** nataly@nullreturn.io

Lead Auditor: Alexander Mazaletskiy **Email:** am@nullreturn.io

Introduction

This document presents the results of a smart contract audit conducted by Null Return for the DZap protocol. The purpose of the audit was to evaluate the security and correctness of DZap's smart contracts prior to deployment. Special attention was given to identifying vulnerabilities that may lead to loss of funds, unauthorized contract interactions, or logical errors in fee distribution, wallet execution, and cross-protocol zapping operations.

About DZap

DZap is a cross-protocol zapping platform designed to facilitate seamless token swaps, liquidity provisioning, and wallet-based transaction execution. It enables users to perform complex operations (e.g., token transfers, approvals, and external contract interactions) in a single transaction, leveraging a modular architecture with secure backend validation. Key features include:

- **Zapping Operations:** Aggregates multiple token interactions (e.g., ERC20 transfers, approvals) into a single transaction for efficiency.
- **Wallet Management:** Supports user-controlled smart wallets with validator-based execution and nonce-protected signatures.
- **Fee and Referral System:** Implements a flexible fee structure with referral incentives, secured by backend checks.
- **Backend Security:** Relies on a trusted backend for data signing, whitelisting, and validation to mitigate on-chain risks.
- **Permit and Permit2 Integration:** Supports gas-efficient token approvals via permit mechanisms.

The contracts are designed for interoperability with various DeFi protocols, with a focus on security, gas efficiency, and user experience.

Project Overview

DZap is designed as a flexible zapping platform built to serve users looking to execute complex DeFi operations efficiently. By aggregating token swaps, approvals, and external contract calls into a single transaction, it reduces gas costs and improves user experience. The protocol supports smart wallets with validator-based execution, enabling secure and customizable transaction flows. Its fee and referral system incentivizes participation while maintaining economic security through backend validation. The architecture relies on a combination of on-chain logic and off-chain backend controls, ensuring robust security for whitelisted adapters, signed data, and user interactions. The use of permit and Permit2 mechanisms enhances gas efficiency for token approvals.

Scope

This audit covered the core components of the DZap protocol, with a focus on the correctness, safety, and robustness of its smart contracts. The analysis was based on the source code provided in the DZapIO/DZapZappingContracts repository at the specified commit.

Field	Details
Repository	https://github.com/DZapIO/DZapZappingContracts
Initial Commit	3d4c6f189758d6bcacda0e2adb059bc357314740
Scope 1	DZapZappingContracts/contracts/zap/DZapZapCore.sol
Scope 2	DZapZappingContracts/contracts/wallet/DZapWalletFactory.sol
Scope 3	DZapZappingContracts/contracts/wallet/DZapWallet.sol
Scope 4	DZapZappingContracts/contracts/wallet/DZapWalletManager.sol
Scope 5	DZapZappingContracts/contracts/wallet/DZapExecutor.sol
Scope 6	DZapZappingContracts/contracts/shared/libraries/LibAsset.sol
Scope 7	DZapZappingContracts/contracts/shared/libraries/LibPermit.sol
Scope 8	DZapZappingContracts/contracts/shared/libraries/LibValidator.sol

Methodology

The goal of this audit was to identify potential vulnerabilities, logical errors, and deviations from best practices in the DZap smart contracts. Our process followed a structured and layered approach, including:

- 1. Manual Code Review** Each file in scope was reviewed manually by experienced security auditors to analyze:
 - Correctness of zapping and wallet execution logic
 - Access control and validator permissions
 - Reentrancy, delegatecall, and frontrunning risks
 - Fee calculation and economic security
 - Backend integration assumptions
- 2. Compliance Verification** The contracts were checked for conformance with relevant Ethereum standards, including:
 - ERC20 and Permit/Permit2 for token interactions
 - Non-standard wallet execution patterns
- 3. Static Analysis & Tooling** We used automated tools (e.g., Slither, custom linters, and symbolic execution engines) to detect:
 - Known vulnerability patterns (e.g., delegatecall, reentrancy)
 - Unsafe arithmetic and overflow risks

- Storage collisions and signature replay issues
4. **Threat Modeling** We examined critical assumptions and attack surfaces relevant to DZap's architecture, including:
- Delegatecall risks in whitelisted adapters
 - Permit and Permit2 frontrunning vulnerabilities
 - Fee and referral system exploits
 - Backend dependency security
 - Wallet execution and validator compromise scenarios
5. **Issue Reporting & Validation** All issues were documented with severity levels (Low, Medium, High). After reporting, the DZap team responded and applied fixes, which we then re-reviewed and validated.

Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Findings

In this section, we document the security issues identified during the audit of DZap, along with their severity levels, status, and relevant recommendations. Each finding includes a description of the issue, potential impact, and suggestions for remediation.

High

No high issues found.

Medium

No medium issues found.

Low

No low issues found.

Informational

No informational issues found.

Severity Levels

Each issue is categorized according to its severity. Here's how each severity level is defined:

- **High:** Critical vulnerabilities that may result in significant loss of funds, unauthorized control, or failure of core zapping and wallet functionality.
- **Medium:** Issues that could potentially cause financial loss or disrupt functionality but are less immediate or impactful than high-severity issues.
- **Low:** Minor issues that have no immediate risk but may affect user experience or the maintainability of the code.
- **Informational:** Non-critical issues such as minor optimizations or suggestions for best practices.

Disclaimer

This audit is not a guarantee of the absence of vulnerabilities or bugs. Security audits are time-boxed and can only identify issues present at the time of the audit. Audits complement other security measures but cannot replace comprehensive testing, code reviews, and ongoing security monitoring. We recommend that multiple audits be conducted, and a bug bounty program be established for continued testing. This report does not provide any investment advice, nor does it guarantee the performance or compliance of the audited project.

Recommendations

Based on our audit of the DZap smart contracts, we provide the following general recommendations to enhance the security and maintainability of the project:

1. **Regular Independent Audits** While this audit covers a significant portion of the code-base, we strongly recommend conducting additional independent audits at key stages of the project's lifecycle. This will help uncover vulnerabilities, such as delegatecall risks or permit frontrunning, that may not have been identified in the initial review.
2. **Continuous Monitoring and Bug Bounty Program** We recommend the implementation of a public bug bounty program to encourage third-party security researchers to identify and report vulnerabilities in zapping operations, wallet execution, or fee handling. Ongoing monitoring of smart contract behavior in production is also essential.

3. **Comprehensive Unit Testing** Expanding test coverage, particularly in areas with complex logic (e.g., permit handling, fee calculations, delegatecall execution), can help identify edge cases and potential vulnerabilities. Ensure that tests are continuously integrated into the development process.
4. **Keep Up-to-Date with EIP Standards** Stay informed of relevant Ethereum Improvement Proposals (EIPs) that could affect the project, particularly those related to ERC20, Permit/Permit2, or wallet standards. Regularly review and refactor the code to conform to evolving standards, especially for gas-efficient approvals and secure external calls.

About Null Return

Null Return is a Web3 security firm specializing in smart contract audits, formal verification, architecture consulting, and pre-deployment reviews. We work closely with builders to uncover and help resolve vulnerabilities before they become problems — combining deep technical knowledge with a hands-on, product-aware approach. We take pride in being fast, transparent, and highly responsive throughout the audit process.

Contact us:

- Website: nullreturn.io
- Telegram: [@nullreturn_io](https://t.me/nullreturn_io)
- Twitter: [@nullreturn_io](https://twitter.com/nullreturn_io)
- LinkedIn: [NullReturn](https://www.linkedin.com/company/NullReturn)